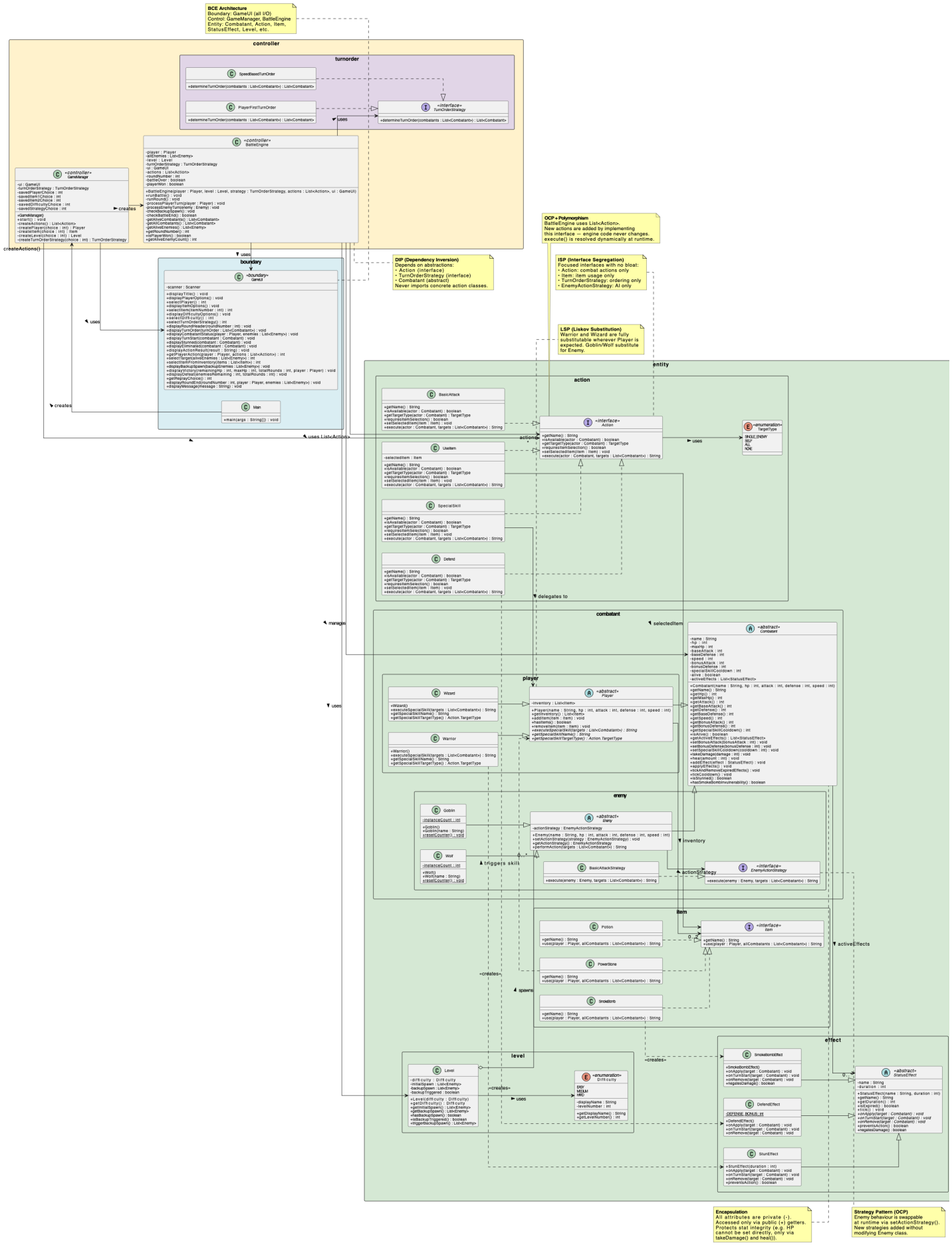
BCE Architecture
Boundary: GameUI (all I/O)
Control: GameManager, BattleEngine
Entity: Combatant, Action, Item, StatusEffect, Level, etc.
controller
turnorder
SpeedBasedTurnOrder
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
PlayerFirstTurnOrder
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
«interface»
TurnOrderStrategy
+determineTurnOrder(combatants : List<Combatant>) : List<Combatant>
uses
«controller»
BattleEngine
-player : Player
-allies : List<Enemy>
-level : Level
-turnOrderStrategy : TurnOrderStrategy
-ui : GameUI
-actions : List<Action>
-roundNumber : int
-battleOver : boolean
-playerWon : boolean
+BattleEngine(player : Player, level : Level, strategy : TurnOrderStrategy, actions : List<Action>, ui : GameUI)
+runBattle() : void
-runRound() : void
-processPlayerTurn(player : Player) : void
-processEnemyTurn(enemy : Enemy) : void
-applyRoundEnd() : void
-checkBattleEnd() : boolean
-getAliveCombatants() : List<Combatant>
-getAllCombatants() : List<Combatant>
-getAliveEnemies() : List<Enemy>
+getRoundNumber() : int
+isPlayerWon() : boolean
+getAliveEnemyCount() : int
«controller»
GameManager
-ui : GameUI
-turnOrderStrategy : TurnOrderStrategy
-savedPlayerChoice : int
-savedItem1Choice : int
-savedItem2Choice : int
-savedDifficultyChoice : int
-savedStrategyChoice : int
+gameManager()
+start() : void
-createActions() : List<Action>
-createPlayer(choice : int) : Player
-createItem(choice : int) : Item
-createLevel(choice : int) : Level
-createTurnOrderStrategy(choice : int) : TurnOrderStrategy
creates
createActions()
uses
boundary
«boundary»
GameUI
-scanner : Scanner
+displayTitle() : void
+displayPlayerOptions() : void
+selectPlayer() : int
+displayItemOptions() : void
+selectItem(itemNumber : int) : int
+displayDifficultyOptions() : void
+selectDifficulty() : int
+selectTurnOrderStrategy() : int
+displayRoundHeader(roundNumber : int) : void
+displayTurnOrder(turnOrder : List<Combatant>) : void
+displayCombatantStatus(player : Player, enemies : List<Enemy>) : void
+displayTurnStart(combatant : Combatant) : void
+displayAlreadyDefending(combatant : Combatant) : void
+displayEliminated(combatant : Combatant) : void
+displayActionResult(result : String) : void
+getPlayerAction(player : Player, actions : List<Action>) : int
+selectTarget(aliveEnemies : List<Enemy>) : int
+selectItemFromInventory(items : List<Item>) : int
+displayBackupSpawn(backupEnemies : List<Enemy>) : void
+displayVictory(remainingHp : int, maxHp : int, totalRounds : int, player : Player) : void
+displayDefeat(enemiesRemaining : int, totalRounds : int) : void
+displayRoundEnd() : void
+displayRoundEnd(roundNumber : int, player : Player, enemies : List<Enemy>) : void
+displayMessage(message : String) : void
Main
+main(args : String[]) : void
DIP (Dependency Inversion)
Depends on abstractions:
• Action (interface)
• TurnOrderStrategy (interface)
• Combatant (abstract)
Never imports concrete action classes.
OCP + Polymorphism
BattleEngine uses List<Action>.
New actions are added by implementing this interface — engine code never changes.
execute() is resolved dynamically at runtime.
ISP (Interface Segregation)
Focused interfaces with no bloat:
• Action: combat actions only
• Item: item usage only
• TurnOrderStrategy: ordering only
• EnemyActionStrategy: AI only
LSP (Liskov Substitution)
Warrior and Wizard are fully substitutable wherever Player is expected. Goblin/Wolf substitute for Enemy.
entity
action
BasicAttack
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
UseItem
-selectedItem : Item
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
SpecialSkill
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
Defend
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
«interface»
Action
+getName() : String
+isAvailable(actor : Combatant) : boolean
+getTargetType(actor : Combatant) : TargetType
+requiresItemSelection() : boolean
+setSelectedItem(item : Item) : void
+execute(actor : Combatant, targets : List<Combatant>) : String
«enumeration»
TargetType
SINGLE_ENEMY
SELF
ALL
NONE
actions
uses
uses List<Action>
delegates to
manages
combatant
selectedItem
«abstract»
Combatant
-name : String
-hp : int
-maxHp : int
-baseAttack : int
-baseDefense : int
-baseSpeed : int
-bonusAttack : int
-bonusDefense : int
-specialSkillCooldown : int
-alive : boolean
-activeEffects : List<StatusEffect>
+Combatant(name : String, hp : int, attack : int, defense : int, speed : int)
+getName() : String
+getHp() : int
+getMaxHp() : int
+getAttack() : int
+getBaseAttack() : int
+getDefense() : int
+getBaseDefense() : int
+getSpeed() : int
+getBonusAttack() : int
+getBonusDefense() : int
+getSpecialSkillCooldown() : int
+isAlive() : boolean
+getActiveEffects() : List<StatusEffect>
+setBonusAttack(bonusAttack : int) : void
+setBonusDefense(bonusDefense : int) : void
+setSpecialSkillCooldown(cooldown : int) : void
+takeDamage(damage : int) : void
+heal(amount : int) : void
+addEffect(effect : StatusEffect) : void
+applyEffects() : void
+removeEffects(effects) : void
+tickCooldown() : void
+isStunned() : boolean
+hasSmokeBombInvulnerability() : boolean
player
Wizard
+Wizard()
+executeSpecialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
Warrior
+Warrior()
+execute(specialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
«abstract»
Player
-inventory : List<Item>
+Player(name : String, hp : int, attack : int, defense : int, speed : int)
+addItem(item : Item) : void
+hasItem() : boolean
+removeItem(item : Item) : void
+executeSpecialSkill(targets : List<Combatant>) : String
+getSpecialSkillName() : String
+getSpecialSkillTargetType() : Action.TargetType
enemy
Goblin
-instanceCount : int
+Goblin()
+Goblin(name : String)
+resetCounter() : void
Wolf
-instanceCount : int
+Wolf()
+Wolf(name : String)
+resetCounter() : void
«abstract»
Enemy
-actionStrategy : EnemyActionStrategy
+Enemy(name : String, hp : int, attack : int, defense : int, speed : int)
+setActionStrategy(strategy : EnemyActionStrategy) : void
+getActionStrategy() : EnemyActionStrategy
+performAction(targets : List<Combatant>) : String
BasicAttackStrategy
+execute(enemy : Enemy, targets : List<Combatant>) : String
«interface»
EnemyActionStrategy
+execute(enemy : Enemy, targets : List<Combatant>) : String
inventory
triggers skill
actionStrategy
item
Potion
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
PowerStone
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
SmokeBomb
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
«interface»
Item
+getName() : String
+use(player : Player, allCombatants : List<Combatant>) : String
activeEffects
«creates»
spawns
effect
SmokeBombEffect
+SmokeBombEffect()
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+negatesDamage() : boolean
DefendEffect
-DEFENSE_BONUS : int
+DefendEffect()
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
StunEffect
+StunEffect(duration : int)
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+preventsAction() : boolean
«abstract»
StatusEffect
-name : String
-duration : int
+StatusEffect(name : String, duration : int)
+getName() : String
+getDuration() : int
+isExpired() : boolean
+tick() : void
+onApply(target : Combatant) : void
+onTurnStart(target : Combatant) : void
+onRemove(target : Combatant) : void
+preventsAction() : boolean
+negatesDamage() : boolean
level
Level
-difficulty : Difficulty
-initialSpawn : List<Enemy>
-backupSpawn : List<Enemy>
-backupTriggered : boolean
+Level(difficulty : Difficulty)
+getDifficulty() : Difficulty
+getInitialSpawn() : List<Enemy>
+getBackupSpawn() : List<Enemy>
+hasBackupSpawn() : boolean
+setBackupTriggered() : void
+hasBeenBackupSpawned() : boolean
«enumeration»
Difficulty
EASY
MEDIUM
HARD
-displayName : String
-levelNumber : int
+getDisplayName() : String
+getLevelNumber() : int
Encapsulation
All attributes are private (-).
Accessed only via public (+) getters.
Protects stat integrity (e.g. HP cannot be set directly, only via takeDamage() and heal()).
Strategy Pattern (OCP)
Enemy behaviour is swappable at runtime via setActionStrategy().
New strategies added without modifying Enemy class.